

Module 3 Activity 6: Applying Selections in Python

Purpose:

Incorporate the use of Select by Attributes and Select by Location in Python scripts as well as the **MakeFeatureLayer** and **GetCount** functions to retrieve additional information.

Instructions:

Any Python environment will work for this exercise.

Part 1 – Select by Attributes:

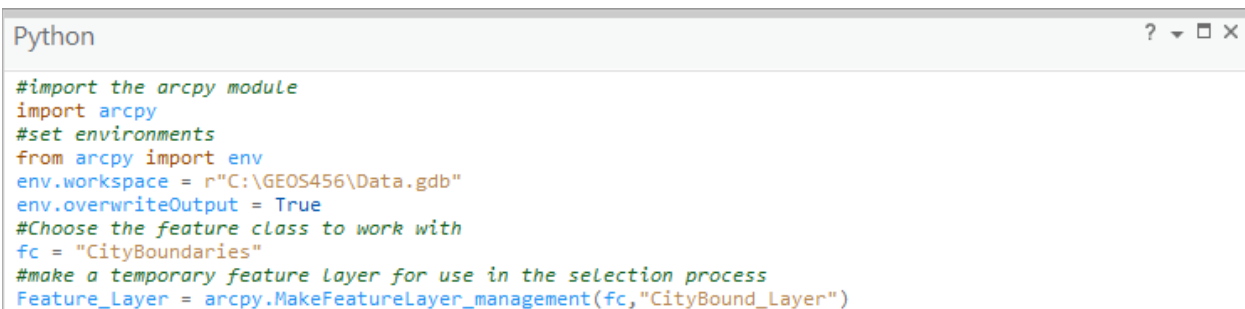
1. Import the **arcpy** module
2. Set the workspace
3. Set the overwrite option to **True**

Ensure the **overwriteOutput** is set to **True**. We will be using the **MakeFeatureLayer** function, which will create a temporary feature stored in memory. If the **overwriteOutput** isn't set, the script will have errors saying the feature already exists.

4. Choose a feature class you want to work with
5. Create a variable called **Feature_Layer** and use this to store the values of the **MakeFeatureLayer** function

The syntax for the **MakeFeatureLayer** function is as follows:

```
MakeFeatureLayer_management (in_features, out_layer, {where_clause}, {workspace}, {field_info})
```



```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
```

6. Use the **AddFieldDelimiters** function to specify which field will be used in the query for the selection

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
```

7. Add the **SelectLayerByAttribute** function to specify which feature class will be selected, what type of selection will be made and the query that the selection will be based on
 - Selection = **NEW_SELECTION**
 - SQL query = **STATUS = NO CITY**

```
Python
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#access the SelectLayerByAttribute function to create a new selection based on a definition query
arcpy.SelectLayerByAttribute_management(Feature_Layer,"NEW_SELECTION", delimitedField + " = 'NO CITY'")
```

8. Next, use the **GetCount** function to print a number of selected features to the screen. The **GetCount** function needs to reference the temporary feature created in **step 5**
9. Create a **print** statement that will tell you how many records have been selected

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#access the SelectLayerByAttribute function to create a new selection based on a definition query
arcpy.SelectLayerByAttribute_management(Feature_Layer,"NEW_SELECTION", delimitedField + " = 'NO CITY'")
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
```

10. Run the script and examine your results

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#access the SelectLayerByAttribute function to create a new selection based on a definition query
arcpy.SelectLayerByAttribute_management(Feature_Layer,"NEW_SELECTION", delimitedField + " = 'NO CITY'")
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
The number of selected records is: 71
```

The processes listed above works very much like the manual process in Pro. A new temporary feature is added to your table of contents. The **MakeFeatureLayer** operates the same way as right clicking your selected data and creating a new layer from selected features. Both of these options create a temporary feature that is stored in memory.

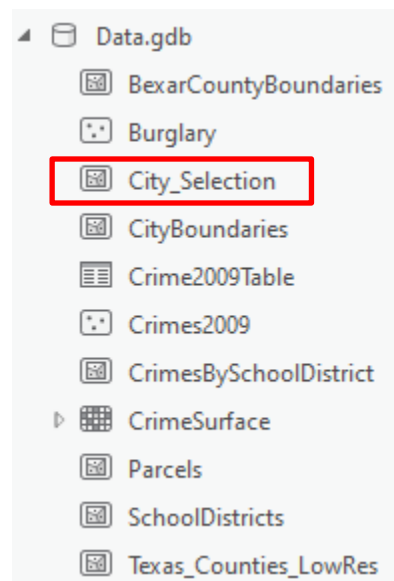
11. Use the **FeatureClassToFeatureClass** function to export the temporary feature layer to a permanent feature class in your geodatabase
12. Use a **print** statement to tell the user that the feature has been successfully created

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#access the SelectLayerByAttribute function to create a new selection based on a definition query
arcpy.SelectLayerByAttribute_management(Feature_Layer,"NEW_SELECTION", delimitedField + " = 'NO CITY'")
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
#export the feature layer to a feature class in the geodatabase
arcpy.FeatureClassToFeatureClass_conversion(Feature_Layer,"C:\GEOS456\Data.gdb", "City_Selection")
#CopyFeatures_management can also work for this process
print ("Successfully created selected feature class")
#consider using GetMessages for geoprocessing operations for clearer results
```

13. Run the script and examine your results

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#Choose the feature class to work with
fc = "CityBoundaries"
#make a temporary feature layer for use in the selection process
Feature_Layer = arcpy.MakeFeatureLayer_management(fc,"CityBound_Layer")
#use the AddFieldDelimiters function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#access the SelectLayerByAttribute function to create a new selection based on a definition query
arcpy.SelectLayerByAttribute_management(Feature_Layer,"NEW_SELECTION", delimitedField + " = 'NO CITY'")
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
#export the feature layer to a feature class in the geodatabase
arcpy.FeatureClassToFeatureClass_conversion(Feature_Layer,"C:\GEOS456\Data.gdb", "City_Selection")
#CopyFeatures_management can also work for this process
print ("Successfully created selected feature class")
#consider using GetMessages for geoprocessing operations for clearer results
The number of selected records is: 71
Successfully created selected feature class
```

14. Check ArcMap or ArcCatalog to ensure the feature class has been created



Part 2 – Select by Location:

1. Import the **arcpy** module
2. Set the workspace
3. Set the overwrite option to **True**
4. Specify which feature class you want to work with as well as a comparison layer
5. Create a variable called **Feature_Layer** and use this to store the values of the **MakeFeatureLayer** function

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#specify the feature class to work with
InputFC = "Crimes2009"
#specify a comparison layer for use in the select by location tool
ComparisonFC = "CityBoundaries"
#make a temporary feature layer
Feature_Layer = arcpy.MakeFeatureLayer_management(InputFC,"Crimes2009_Layer")
```

NOTE: When selecting an input feature class and a comparison layer, you must be aware of the spatial relationships between them. If the relationship isn't satisfied, the tool will not run. For example, you cannot have the tool run a selection that finds polygons that are within points.

6. Call the select by location function that will select the **InputFC** that is **COMPLETELY_WITHIN** the **ComparisonFC**

The **SelectLayerByLocation** function syntax is below

```
SelectLayerByLocation_management (in_layer, {overlap_type}, {select_features}, {search_distance}, {selection_type},
{invert_spatial_relationship})
```

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#specify the feature class to work with
InputFC = "Crimes2009"
#specify a comparison layer for use in the select by location tool
ComparisonFC = "CityBoundaries"
#make a temporary feature layer
Feature_Layer = arcpy.MakeFeatureLayer_management(InputFC,"Crimes2009_Layer")
#access the Select Layer By Location function to create a new selection based on an overlap type
arcpy.SelectLayerByLocation_management(Feature_Layer,"COMPLETELY_WITHIN", ComparisonFC)
```

7. Use the same **GetCount** function and **print** statement that was used in **Part 1**

Instead of using the **FeatureClassToFeatureClass** function, we will use the **CopyFeatures** function. Both functions work similarly, so you can choose which one you would like to use. The **CopyFeatures** function will honour any selections present in the feature, in this case it will honour the selection created by the **SelectLayerByLocation** function.

CopyFeatures function is as follows:

```
CopyFeatures_management (in_features, out_feature_class, {config_keyword}, {spatial_grid_1}, {spatial_grid_2},
{spatial_grid_3})
```

8. Call on the **CopyFeatures** function and specify the appropriate parameters
9. Name the output feature class **CrimesWITHINCity**
10. Use a **print** statement to tell the user that the feature has been successfully created

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#specify the feature class to work with
InputFC = "Crimes2009"
#specify a comparison layer for use in the select by location tool
ComparisonFC = "CityBoundaries"
#make a temporary feature layer
Feature_Layer = arcpy.MakeFeatureLayer_management(InputFC,"Crimes2009_Layer")
#access the Select Layer By Location function to create a new selection based on an overlap type
arcpy.SelectLayerByLocation_management(Feature_Layer,"COMPLETELY_WITHIN", ComparisonFC)
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
#export the feature layer to a feature class in the geodatabase use the Copy Features tool
arcpy.management.CopyFeatures(Feature_Layer, "CrimesWITHINCity")
print ("Successfully created selected feature class")
```

11. Run the script and examine your results

```
Python
#import the arcpy module
import arcpy
#set environments
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
env.overwriteOutput = True
#specify the feature class to work with
InputFC = "Crimes2009"
#specify a comparison layer for use in the select by location tool
ComparisonFC = "CityBoundaries"
#make a temporary feature layer
Feature_Layer = arcpy.MakeFeatureLayer_management(InputFC,"Crimes2009_Layer")
#access the Select Layer By Location function to create a new selection based on an overlap type
arcpy.SelectLayerByLocation_management(Feature_Layer,"COMPLETELY_WITHIN", ComparisonFC)
#use the Get Count function to return a number of features selected
#the Get Count needs to reference the temporary layer created using the make feature layer function
Select_Count = arcpy.GetCount_management(Feature_Layer)
print ("The number of selected records is: " + str(Select_Count))
#export the feature layer to a feature class in the geodatabase use the Copy Features tool
arcpy.management.CopyFeatures(Feature_Layer, "CrimesWITHINCity")
print ("Successfully created selected feature class")
The number of selected records is: 178349
Successfully created selected feature class
```

12. Add the new feature class to Pro and check the attributes to ensure is worked as planned

Untitled - ArcGIS Pro

CrimesWITHINCity X

Field: Add Calculate Selection: Select By Attributes Zoom To Switch Clear Delete Copy

	OBJECTID *	Shape *	CASE	LOCATION	DIST	SVCAREA	SPLTTDT	SPLTTM	HR	DOW	SHIFT	OFFCODE	OFFDESC	ARCCODE
1	1	Point	9069434301	1533 AUSTIN HWY	3320	North	9/15/2009	12/30/1899	0	Tue	C	220301	BURGL-FORCED ENT...	YBURGBLD
2	2	Point	9069842401	4511 HOLLYRIDGE ST	5230	West	9/15/2009	12/30/1899	17	Tue	B	890901	NONE	OTHER
3	3	Point	9069842501	466 MORNINGVIEW...	4220	East	9/15/2009	12/30/1899	18	Tue	B	000056	WNTD PERS	OTHER
4	4	Point	9069843601	711 MAYFIELD BV W	6140	South	9/15/2009	12/30/1899	13	Tue	A	239904	LARCENY	YLARCENY
5	5	Point	9069844401	4801 GUS ECKERT	7150	Prue	9/15/2009	12/30/1899	17	Tue	B	220001	BURGLARY	YBURGVEH
6	6	Point	9069844801	4002 BELLE GROVE ST	7220	Prue	9/15/2009	12/30/1899	18	Tue	B	220201	BURGL-FORCED ENT...	YBURGHAB
7	7	Point	9069844901	2929 RIGSBY AV	4220	East	9/15/2009	12/30/1899	18	Tue	B	131352	SIMPLE ASSLT	YFAMVIOL
8	8	Point	9069845001	1924 MONTANA ST	4150	East	9/15/2009	12/30/1899	18	Tue	B	131343	SIMPLE ASSLT	YFAMVIOL
9	9	Point	9069845201	4911 CELTIC CORNER	4180	East	9/15/2009	12/30/1899	18	Tue	B	530950	PUBLIC PEACE	OTHER
10	10	Point	9069845701	122 BRIDLE RIDGE	5160	West	9/15/2009	12/30/1899	18	Tue	B	890901	NONE	OTHER
11	11	Point	9069845801	2313 NW MII ITALY...	7230	Prue	9/15/2009	12/30/1899	18	Tue	R	220001	BURGLARY	YBURGVFH

0 of 178,349 selected

Filters: 100%

Next Steps:

1. Try using the both the select by attributes and select by location tools with different selection types and queries
2. Incorporate the use of a geoprocessing tool once features have been selected